

Creating and inserting elements in JavaScript

Now we will learn how to create new DOM elements through JavaScript, and then add them to the page. The `createElement` method is designed for this aim. Name of the tag to be created should be passed as the parameter of this method.

If we write the result of `createElement` into a variable, then this variable will contain such an element as if we got it through `querySelector`.

The only difference is that our element will not be placed on the page. And so we can change its text, attributes, hang event handlers and eventually place it on the page.

The `appendChild` method is used to place a new element on the page. This method should be applied to the element *in which* we want to place our element. And our new element created earlier through `createElement` should be passed as the method parameter.

Let's look at a practical example. Suppose we have a div and a few paragraphs in it:

```
1 <div id="parent">
2   <p>1</p>
3   <p>2</p>
4   <p>3</p>
5 </div>
```

Let's create a paragraph, set the text for it and place it on the page at the end of the `#parent` block:

```
1 let parent = document.querySelector('#parent');
2
3 let p = document.createElement('p');
4 p.textContent = '!';
5
6 parent.appendChild(p);
```

Result of code execution:

```
1 <div id="parent">
2   <p>1</p>
3   <p>2</p>
4   <p>3</p>
5   <p>!</p>
6 </div>
```

No 1

©jsPmMnCIE

Given an `ol`:

```
1 <ol id="elem"></ol>
```

Insert a `li` at the end of it with the text 'item'.

No 2

©jsPmMnCIE

Given an `ol` and a button:

```
1 <ol id="elem"></ol>
2 <button id="button">click me</button>
```

Make it so that when you click on the button, a `li` is added to the end of the list with the text 'item'.

Binding events on insertion of elements

Now, when inserting elements, let's bind event handlers to them.

Let, for example, we have such a parent element:

```
1 | <div id="parent">
2 |   <p>1</p>
3 |   <p>2</p>
4 |   <p>3</p>
5 | </div>
```

Let's add another paragraph to the end of this parent by attaching a click handler to it:

```
1 | let parent = document.querySelector('#parent');
2 |
3 | let p = document.createElement('p');
4 | p.textContent = '!';
5 |
6 | p.addEventListener('click', function() {
7 |   console.log(this.textContent); // displays the text of the paragraph on click
8 | });
9 |
10 | parent.appendChild(p);
```

No 1

©jsPmMnIBE

Given an `ol` and a button. Make it so that when you click on the button, the tag `li` is added to the end of `ol`. Make it so that when you click on any of the added `li`, an exclamation mark is written to the end of its text.

Creating elements in a loop in JavaScript

In the previous lessons, we created one new element. Let's now make it so that new elements are created in a loop. For example, let's say we have a parent div like this:

```
1 | <div id="parent"></div>
```

Let's run a loop that adds 9 new paragraphs to the end of our div:

```
1 | let parent = document.querySelector('#parent');
2 |
3 | for (let i = 1; i <= 9; i++) {
4 |   let p = document.createElement('p');
5 |   p.textContent = '!';
6 |
7 |   parent.appendChild(p);
8 | }
```

No 1

©jsPmMnLEC

Given an empty `ul`. Run a loop that will insert 10 `li` tags into it. Make sure that the text of each `li` is its ordinal number.

Attaching handlers in a loop in JavaScript

In the previous lesson, we learned how to create new elements in a loop. Let's now hang event handlers when creating new elements.

Let us again have such a parent div:

```
1 | <div id="parent"></div>
```

Let's run a loop that will add 9 new paragraphs to the end of our div, attaching a click handler to them:

```

1 let parent = document.querySelector('#parent');
2
3 for (let i = 1; i <= 9; i++) {
4   let p = document.createElement('p');
5   p.textContent = '!';
6
7   // Attaching a click handler:
8   p.addEventListener('click', function() {
9     console.log(this.textContent);
10  });
11
12  parent.appendChild(p);
13 }

```

No 1

©jsPmMnLBH

Given a div. Run a loop that will add 5 inputs to our div. Let there be a paragraph. Make it so that each of the new inputs writes its text into a paragraph when focus is lost.

Removing elements in JavaScript

Let's now learn how to remove elements. To do this, use the `remove` method. Let's look at an example of how this is done. Let's have paragraphs:

```

1 <p>elem 1</p>
2 <p>elem 2</p>
3 <p>elem 3</p>
4 <p>elem 4</p>
5 <p>elem 5</p>

```

Let's make it so that any paragraph is removed by clicking on it:

```

1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', function() {
5     elem.remove();
6   });
7 }

```

No 1

©jsPmMnER

Given the following code:

```

1 <ul>
2   <li>1</li>
3   <li>2</li>
4   <li>3</li>
5 </ul>

```

Make it so that any `li` is removed by clicking on it.

No 2

©jsPmMnER

Given the following code:

```

1 <ul id="parent">
2   <li>1</li>
3   <li>2</li>
4   <li>3</li>
5 </ul>

```

```
6 |  
7 | <input type="submit" id="button">
```

Make it so that on each click on the button, the last element from `#parent` is removed.

Insertion of elements before or after in JavaScript

Learn the theory at the following links:

[append](#) [prepend](#)

append method

The `append` method allows you to insert another element at the end of an element. The method takes an element as a parameter, usually created with `createElement`, or a string. You can add several elements or strings at once by listing them separated by commas.

Syntax

```
1 | parent.append(element or string);
```

Example

Let's create a paragraph, set its text and place it on a page at the end of the `#parent` block:

```
1 | <div id="parent">  
2 |   <p>1</p>  
3 |   <p>2</p>  
4 |   <p>3</p>  
5 | </div>  
  
1 | let parent = document.querySelector('#parent');  
2 |  
3 | let p = document.createElement('p');  
4 | p.textContent = '!';  
5 |  
6 | parent.append(p);
```

The code execution result:

```
1 | <div id="parent">  
2 |   <p>1</p>  
3 |   <p>2</p>  
4 |   <p>3</p>  
5 |   <p>!</p>  
6 | </div>
```

prepend method

The `prepend` method allows you to insert another element at the beginning of an element. The method takes an element as a parameter, usually created with `createElement`, or a string. You can add several elements or strings at once by listing them separated by commas.

Syntax

```
1 | parent.prepend(element or string);
```

Example

Let's create a paragraph, set its text and place it on a page at the beginning of the `#parent` block:

```
1 <div id="parent">
2   <p>1</p>
3   <p>2</p>
4   <p>3</p>
5 </div>
```

```
1 let parent = document.querySelector('#parent');
2
3 let p = document.createElement('p');
4 p.textContent = '!';
5
6 parent.prepend(p);
```

The code execution result:

```
1 <div id="parent">
2   <p>!</p>
3   <p>1</p>
4   <p>2</p>
5   <p>3</p>
6 </div>
```

No 1

© jsPmMnIE

Given an `ul`:

```
1 <ul id="elem">
2   <li>text</li>
3   <li>text</li>
4   <li>text</li>
5   <li>text</li>
6   <li>text</li>
7 </ul>
```

Insert an element with the text 'start' at the beginning, and an element with the text 'finish' at the end of it.

Insert elements before JavaScript elements

Learn the theory at the following links:

[insertBefore](#)

insertBefore method

The `insertBefore` method allows you to insert an element before another element. It most commonly used after creating an element with `createElement`. The method is applied to the parent of the element before which the insertion will occur.

Syntax

```
1 parent.insertBefore(element, node before which to insert);
```

Example

Create a paragraph and place it before the second paragraph:

```
1 <div id="parent">
2   <p>elem 1</p>
3   <p id="before">elem 2</p>
4   <p>elem 3</p>
5 </div>

1 let parent = document.querySelector('#parent');
2 let before = document.querySelector('#before');
3
4 let p = document.createElement('p');
5 p.textContent = '!';
6
7 parent.insertBefore(p, before);
```

The code execution result:

```
1 <div id="parent">
2   <p>elem 1</p>
3   <p>!</p>
4   <p>elem 2</p>
5   <p>elem 3</p>
6 </div>
```

No 1

©jsPmMnIB

Given the element #elem:

```
1 <ul id="parent">
2   <li>text</li>
3   <li>text</li>
4   <li id="elem">text</li>
5   <li>text</li>
6   <li>text</li>
7 </ul>
```

Insert an element before this element with the text 'new'.

No 2

©jsPmMnIB

Modify the previous task so that clicking on the inserted element causes '!' to be added to the end of its text.

Adjacent insertion of elements in JavaScript

Learn the theory at the following links:

[insertAdjacentElement](#)

insertAdjacentElement method

The `insertAdjacentElement` method allows you to insert an element anywhere on a page. Most commonly used after creating an element with `createElement`. The code is inserted relative to the *target element*. You can insert before the target element (beforeBegin insertion method), after it (afterEnd insertion method), and also at the beginning (afterBegin insertion method) or at the end (beforeEnd insertion method) of the target element.

Syntax

```
1 | target element.insertAdjacentElement(insertion method, code to insert);
```

Example . beforeBegin method

Let a target element be the `#target` element. We insert a new paragraph before it:

```
1 | <div id="target">
2 |   <p>elem</p>
3 | </div>

1 | let p = document.createElement('p');
2 | p.textContent = '!!!';
3 |
4 | let target = document.querySelector('#target');
5 | target.insertAdjacentElement('beforeBegin', p);
```

The code execution result:

```
1 | <p>!!!</p>
2 | <div id="target">
3 |   <p>elem</p>
4 | </div>
```

Example . afterEnd method

Example . afterBegin method

Example . beforeEnd method

№1

©jsPmMnIA

Given an element:

```
1 | <div id="elem" style="border: 1px solid red;">
2 |   <p>text</p>
3 |   <p>text</p>
4 |   <p>text</p>
5 | </div>
```

Insert a paragraph with the text '!!!' before it.

№2

©jsPmMnIA

Given an element:

```
1 | <div id="elem" style="border: 1px solid red;">
2 |   <p>text</p>
3 |   <p>text</p>
4 |   <p>text</p>
5 | </div>
```

Insert a paragraph with the text '!!!' after it.

Given an element:

```
1 <div id="elem" style="border: 1px solid red;">
2   <p>text</p>
3   <p>text</p>
4   <p>text</p>
5 </div>
```

Insert a paragraph with the text '!!!' at the beginning of it.

Given an element:

```
1 <div id="elem" style="border: 1px solid red;">
2   <p>text</p>
3   <p>text</p>
4   <p>text</p>
5 </div>
```

Insert a paragraph with the text '!!!' at the end of it.

Adjacent tag insertion in JavaScript

Learn the theory at the following links:

[insertAdjacentHTML](#)

insertAdjacentHTML method

The `insertAdjacentHTML` method allows you to insert a line of HTML code anywhere on a page. The code is inserted relative to the *target element*. You can insert before the target element ('beforeBegin' insertion method), after it ('afterEnd' insertion method), and also at the beginning ('afterBegin insertion method ') or at the end (the 'beforeEnd' insertion method) of the target element.

Syntax

```
1 target element.insertAdjacentHTML(insertion method, code to insert);
```

Example . beforeBegin method

Let a target element be the `#target` element. Insert a new paragraph before it:

```
1 <div id="target">
2   <p>elem</p>
3 </div>

1 let target = document.querySelector('#target');
2 target.insertAdjacentHTML('beforeBegin', '<p>!</p>');
```

The code execution result:

```
1 <p>!</p>
2 <div id="target">
3   <p>elem</p>
4 </div>
```


Example . afterEnd method

Example . afterBegin method

Example . beforeEnd method

№ 1

©jsPmMnIAT

Given an element:

```
1 <div id="elem" style="border: 1px solid red;">
2   <p>text</p>
3   <p>text</p>
4   <p>text</p>
5 </div>
```

Given an element. Insert the following tag before it:

```
1 <div class="www"><p>text</p><p>text</p><input></div>
```

Cloning elements in JavaScript

In this lesson, we will learn how to get copies of elements. It will be possible to work with these copies, as with ordinary elements - change them and insert them into the right place on the page. The process of making copies of elements is called *cloning*.

You can clone an element using the `cloneNode` method. True or false must be passed as a parameter of this method. If true is passed, then the element is cloned completely, along with all attributes and child elements, and if false, only the element itself is cloned.

Let's look at an example. Let's say we have this code:

```
1 <div id="parent">
2   <div class="elem">
3     <p>first paragraph</p>
4     <p>second paragraph</p>
5   </div>
6 </div>
```

Make a copy of the block with the `elem` class and paste it at the end of the `#parent` block:

```
1 let parent = document.querySelector('#parent');
2 let elem = parent.querySelector('.elem');
3
4 let clone = elem.cloneNode(true);
5 parent.appendChild(clone);
```

The result will be the following:

```
1 <div id="parent">
2   <div class="elem">
3     <p>first paragraph</p>
4     <p>second paragraph</p>
5   </div>
6   <div class="elem">
7     <p>first paragraph</p>
8     <p>second paragraph</p>
9   </div>
10 </div>
```

Given an input. Given a button. Clone this input by pressing the button.

Checking elements in JavaScript

Learn the theory at the following links:

[matches](#) [contains](#)

matches method

The `matches` method allows you to check if an element matches the specified CSS selector.

Syntax

```
1 | element.matches('selector');
```

Example

We check if our element is the paragraph with the class `www`:

```
1 | <p id="elem" class="www"></p>

1 | let elem = document.querySelector('#elem');
2 | console.log(elem.matches('p.www'));
```

The code execution result:

```
1 | true
```

contains method

The `contains` method allows you to check if one element contains another within itself. The method parameter passes an element that will be checked for being inside the element to which the method was applied.

Syntax

```
1 | parent.contains(element);
```

Example

Check if the paragraph `#child` is in the `#parent` block:

```
1 | <div id="parent">
2 |   <p id="child"></p>
3 | </div>

1 | let parent = document.querySelector('#parent');
2 | let child = document.querySelector('#child');
3 |
4 | let contains = parent.contains(child);
5 | console.log(contains);
```

The code execution result:

```
1 | true
```

No 1

⊗jsPmMnECh

Given an element:

```
1 | <div id="elem" class="www"></div>
```

Check if this element is an element with `www` class.

No 2

⊗jsPmMnECh

Given an element. Check if this element is a paragraph.

No 3

⊗jsPmMnECh

Given two variables `elem1` and `elem2` containing two elements:

```
1 | <div id="elem1">
2 |   <p id="elem2"></p>
3 | </div>
```

```
1 | let elem1 = document.querySelector('#elem1');
2 | let elem2 = document.querySelector('#elem2');
```

Check if an element from `elem2` is a child of an element from `elem1`.